



Politechnika
Śląska



UCZELNIA
BADAWCZA
INICJATYWA DOSKONAŁOŚCI

Raport końcowy

Rodzaj zajęć

Projekt

Przedmiot

Systemy Mikroprocesorowe i Wbudowane

Rok akademicki		Miasto	Kierunek	Semestr	Prowadzący	Grupa	Sekcja
2025/2026		G	INF	5	GB	5	10
Planowany termin wykonywania ćwiczenia			Faktyczny termin wykonania ćwiczenia				
Data		Godzina	Data			Godzina	
02.02.2026		14:00					
Numer ćwiczenia	Temat ćwiczenia						
	Układ sterowania oświetleniem LED						

Skład sekcji		
	Imię i nazwisko	Uwagi
1	Mateusz Smuda	

1. Temat projektu i opis założeń z opisem funkcji urządzenia.

Temat:

Układ sterowania LED z czujnikiem ruchu, czujnikiem temperatury oraz czujnikiem światła

Założenia projektu:

- Stworzenie układu sterującego taśmą LED WS2812B.
- Wykorzystanie czujników do automatycznej regulacji zachowania oświetlenia.
- Zmiana koloru LED w zależności od temperatury (DHT22).
- Zmiana jasności w zależności od natężenia światła (fotorezystor).
- Włączanie oświetlenia po wykryciu ruchu przez czujnik LD2410C.
- Dwa tryby pracy: automatyczny i manualny.
- Sterowanie zdalne przez aplikację webową hostowaną na ESP32.

2. Analiza zadania z uzasadnieniem wyboru elementów elektronicznych i narzędzi użytych do realizacji projektu.

Celem projektu jest stworzenie inteligentnego systemu oświetlenia LED, który łączy funkcje automatycznej regulacji światła z możliwością zdalnego sterowania. Układ zbiera informacje o temperaturze, natężeniu światła oraz wykryciu ruchu, a następnie wykorzystuje te dane do odpowiedniego dostosowania barwy i jasności taśmy LED WS2812B. System przewiduje dwa tryby pracy: automatyczny i manualny. W trybie automatycznym oświetlenie reaguje samoczynnie na dane z czujników, natomiast w trybie manualnym użytkownik sam wybiera parametry światła za pomocą prostej aplikacji webowej. ESP32 pełni rolę jednostki centralnej, obsługuje komunikację z czujnikami oraz generuje sygnały sterujące LED-ami, a dzięki wbudowanemu Wi-Fi umożliwia wygodne sterowanie z poziomu przeglądarki.

Do realizacji projektu wybrano mikrokontroler ESP32 ze względu na:

- wbudowany moduł Wi-Fi,
- dużą liczbę wejść/wyjść,
- wysoką wydajność,
- szerokie wsparcie bibliotek programistycznych.

Taśma LED WS2812B została wybrana ze względu na możliwość indywidualnego adresowania diod, co pozwala na precyzyjną kontrolę koloru i jasności przy użyciu jednego pinu sterującego.

Czujnik LD2410C wykorzystuje technologię radarową, co umożliwia wykrywanie obecności człowieka niezależnie od warunków oświetleniowych, wybrany został przez swoją precyzję pomiarów i możliwość wygodnego dostosowywania parametrów.

Czujnik DHT22 umożliwia pomiar temperatury został on wybrany za dokładność i możliwość łatwego montażu.

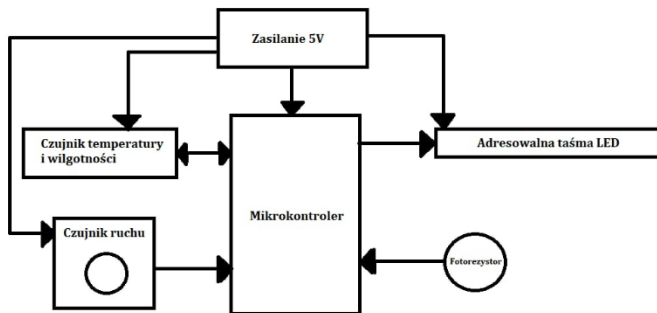
Moduł fotorezystora pozwala na ocenę poziomu oświetlenia otoczenia został wybrany z powodu łatwości obsługi.

Do programowania wykorzystano środowisko Arduino IDE, które umożliwia szybkie prototypowanie oraz łatwą integrację bibliotek obsługujących zastosowane komponenty.

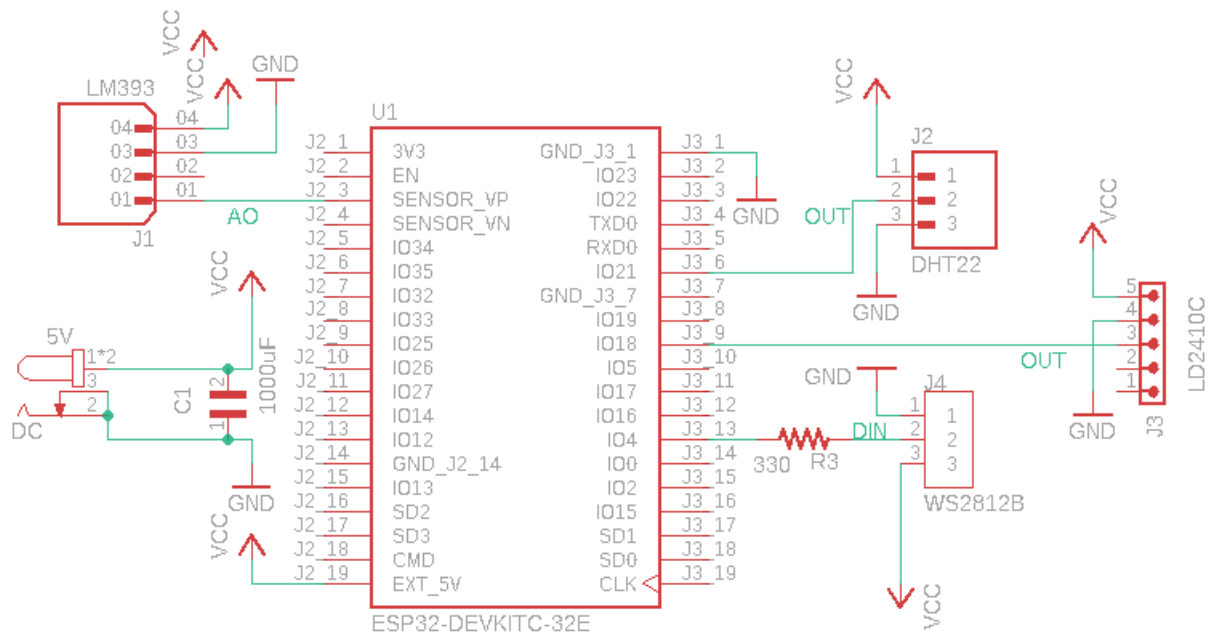
3. Specyfikacja wewnętrzna:

a. Schemat blokowy i ideowy urządzenia.

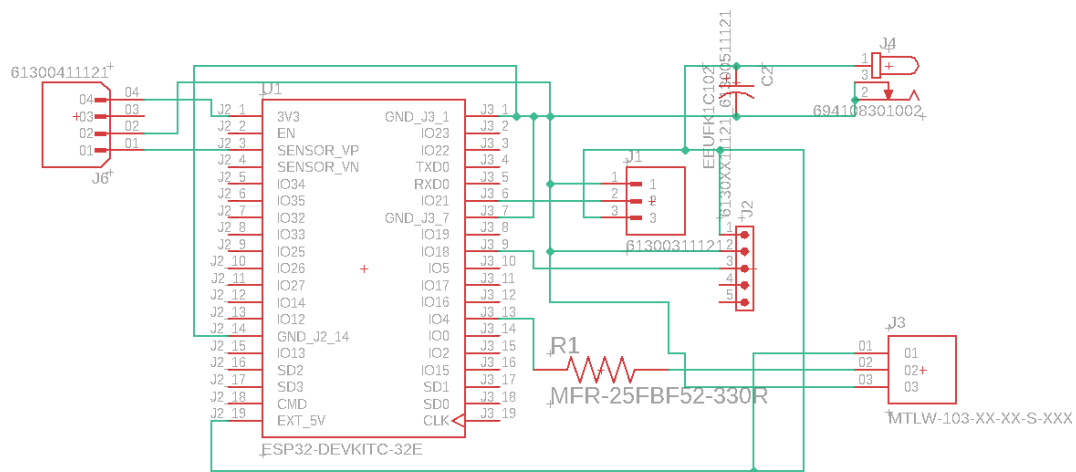
Schemat blokowy:



Schemat ideowy z podpisanymi modułami bez prowadzenia długich ścieżek w celu lepszej widoczności i sprawdzenia poprawności układu:



Schematu użyty do zaprojektowania płytki PCB ze ścieżkami:



b. Opis funkcji poszczególnych bloków układu, szczegółowy opis działania ważniejszych elementów układu.

ESP32 – jednostka centralna systemu, przetwarza dane z czujników, steruje diodami LED oraz obsługuje komunikację Wi-Fi.

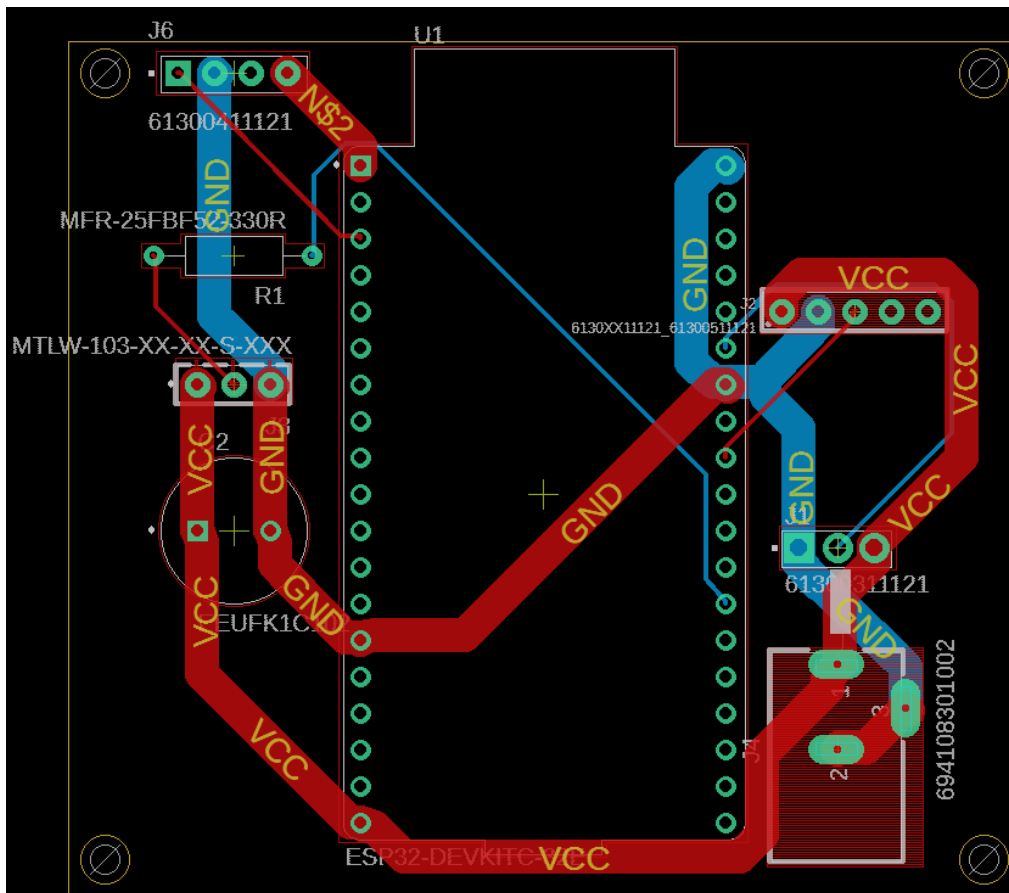
LD2410C – czujnik ruchu, generuje sygnał informujący o wykryciu ruchu.

DHT22 – czujnik temperatury, dostarcza danych do regulacji koloru LED.

Fotorezystor – mierzy poziom światła otoczenia, na podstawie którego regulowana jest jasność LED.

WS2812B – element generujący światło zgodnie z pomiarami czujników.

c. Schemat montażowy obejmujący projekt płytki drukowanej, rozmieszczenie elementów na płycie.



Lista elementów:

ESP32-DEVKITC-32E

694108301002 – złącze zasilania DC 2.5mm/5.5mm

Złącza goldpin męskie 3 pin/4 pin oraz 5 pin o rastrze 2.54mm do podpięcia czujników(DNT22,LD2410C,moduł fotorezystor) oraz adresowalnego paska LED WS2812B

Rezystor 330Ohm

Kondensator 1000uF

d. Algorytm oprogramowania urządzenia

Algorytm działania oprogramowania opiera się na pętli głównej programu, w której mikrokontroler ESP32 cyklicznie obsługuje komunikację sieciową, odczytuje dane z czujników oraz steruje taśmą LED. Po uruchomieniu urządzenia następuje inicjalizacja mikroprocesora, czujników oraz połączenia Wi-Fi. Następnie uruchamiany jest serwer WWW, który udostępnia panel sterowania użytkownikowi.

W głównej pętli programu realizowane są następujące kroki:

1. Obsługa żądań HTTP z aplikacji webowej.
2. Cykliczny odczyt temperatury, natężenia światła oraz stanu czujnika ruchu.

3. Sprawdzenie aktualnego trybu pracy (automatyczny lub manualny).
4. W trybie automatycznym – obliczenie koloru i jasności LED na podstawie danych z czujników.
5. W trybie manualnym – ustawienie parametrów LED zgodnie z poleceniami użytkownika.
6. Aktualizacja stanu taśmy LED.

Główny kod programu do sterowania:

```
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_NeoPixel.h>
#include <DHT.h>

#include "index.h"

const char* ssid      = "HUAWEI Mate 10 Pro";
const char* password = "59befa2de9a3";

#define LED_PIN      4
#define RADAR_PIN    18
#define DHT_PIN      21
#define LDR_PIN      36

#define NUM_PIXELS   60
#define DHT_TYPE      DHT22

Adafruit_NeoPixel pixels(NUM_PIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);
DHT dht(DHT_PIN, DHT_TYPE);
WebServer server(80);

const unsigned long CZAS_SWIECENIA = 10000;
unsigned long ostatniRuchCzas = 0;
unsigned long ostatniOdczytSensorow = 0;

bool trybAuto = true;
bool manualState = false;
uint32_t manualColor = 0;
int manualBrightness = 50;

float temperatura = 0.0;
int jasnocLDR = 0;
bool wykrytoRuch = false;

void setup() {
    Serial.begin(9600);

    pinMode(RADAR_PIN, INPUT_PULLDOWN);
    pinMode(LDR_PIN, INPUT);
```

```

dht.begin();
pixels.begin();
pixels.setBrightness(100);
pixels.show();

Serial.print("łączenie z WiFi");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nPołączono!");
Serial.print("Adres IP strony: http://");
Serial.println(WiFi.localIP());

server.on("/", []() {
    server.send(200, "text/html", index_html);
});

server.on("/status", []() {
    String json = "{";
    json += "\"temp\":" + String(temperatura) + ",";
    json += "\"ldr\":" + String(jasnoscLDR) + ",";
    json += "\"radar\":" + String(wykrytoRuch) + ",";
    json += "\"modeAuto\":" + String(trybAuto);
    json += "}";
    server.send(200, "application/json", json);
});

server.on("/set", handleControl);

server.begin();
}

void loop() {
    server.handleClient();

    unsigned long aktualnyCzas = millis();

    if (aktualnyCzas - ostatniOdczytSensorow > 2000) {
        float t = dht.readTemperature();
        if (!isnan(t)) temperatura = t;

        jasnoscLDR = analogRead(LDR_PIN);

        wykrytoRuch = (digitalRead(RADAR_PIN) == HIGH);

        ostatniOdczytSensorow = aktualnyCzas;
    }
}

```

```

    }

    if (trybAuto) {
        logikaAutomatyczna(aktualnyCzas);
    } else {
        logikaManualna();
    }
}

void logikaAutomatyczna(unsigned long czas) {
    int stanRadaru = digitalRead(RADAR_PIN);

    if (stanRadaru == HIGH) {
        ostatniRuchCzas = czas;
    }

    if (czas - ostatniRuchCzas < CZAS_SWIECENIA) {

        uint32_t kolor;
        if (temperatura < 20.0)      kolor = pixels.Color(0, 0, 255);
        else if (temperatura > 26.0) kolor = pixels.Color(255, 0, 0);
        else                        kolor = pixels.Color(0, 255, 0);

        int jasnoscpwm = map(jasnoscldr, 0, 4095, 255, 10);
        jasnoscpwm = constrain(jasnoscpwm, 10, 255);

        pixels.setBrightness(jasnoscpwm);
        fillStrip(kolor);

    } else {
        pixels.clear();
        pixels.show();
    }
}

void logikaManualna() {
    pixels.setBrightness(manualBrightness);

    if (manualState) {
        if (manualColor == 0) manualColor = pixels.Color(255, 255, 255);
        fillStrip(manualColor);
    } else {
        pixels.clear();
        pixels.show();
    }
}

void handleControl() {
    if (server.hasArg("mode")) {

```



```

    String m = server.arg("mode");
    if (m == "auto") trybAuto = true;
    if (m == "manual") trybAuto = false;
}

if (server.hasArg("manual")) {
    manualState = (server.arg("manual").toInt() == 1);
}

if (server.hasArg("brightness")) {
    manualBrightness = server.arg("brightness").toInt();
}

if (server.hasArg("color")) {
    String hex = server.arg("color");
    long number = strtol(hex.c_str(), NULL, 16);
    int r = number >> 16;
    int g = (number >> 8) & 0xFF;
    int b = number & 0xFF;
    manualColor = pixels.Color(r, g, b);
}
server.send(200, "text/plain", "OK");
}

void fillStrip(uint32_t color) {
    for(int i=0; i<NUM_PIXELS; i++) {
        pixels.setPixelColor(i, color);
    }
    pixels.show();
}

```

Kod interfejsu strony internetowej:

```

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Smart Schody - Panel</title>
    <style>
        body { font-family: 'Segoe UI', Arial, sans-serif; background-color:
#1a1a1a; color: #f0f0f0; text-align: center; margin: 0; padding: 20px; }
        h1 { color: #00d2ff; margin-bottom: 5px; }
        h3 { color: #aaa; margin-top: 0; }

        .card { background-color: #2d2d2d; max-width: 400px; margin: 20px auto;
padding: 25px; border-radius: 15px; box-shadow: 0 10px 20px rgba(0,0,0,0.5); }

```

```

    .sensor-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 15px;
margin-bottom: 20px; }
    .sensor-box { background: #383838; padding: 15px; border-radius: 10px; }
    .val { font-size: 1.8rem; font-weight: bold; color: #fff; }
    .label { font-size: 0.9rem; color: #888; }

    .btn-group { display: flex; justify-content: center; gap: 10px; margin:
20px 0; }
    button { padding: 12px 20px; font-size: 16px; border: none; border-radius:
8px; cursor: pointer; transition: 0.2s; font-weight: bold; width: 100%; }

    .active { border: 2px solid white; transform: scale(1.05); }
    .btn-auto { background-color: #007bff; color: white; }
    .btn-man { background-color: #ffc107; color: black; }

    .manual-controls { background: #444; padding: 15px; border-radius: 10px;
margin-top: 15px; }
    .toggle-btn { padding: 15px; width: 100%; margin-bottom: 10px; font-size:
1.2rem; }
    .on { background-color: #28a745; color: white; }
    .off { background-color: #dc3545; color: white; }

    input[type="color"] { width: 100%; height: 50px; border: none; cursor:
pointer; border-radius: 5px; }
    input[type="range"] { width: 100%; margin: 10px 0; }
</style>
</head>
<body>

<h1>Smart Schody</h1>
<h3>Panel Sterowania</h3>

<div class="card">
  <div class="sensor-grid">
    <div class="sensor-box">
      <div class="label">Temperatura</div>
      <div class="val"><span id="temp">--</span> °C</div>
    </div>
    <div class="sensor-box">
      <div class="label">Jasność (LDR)</div>
      <div class="val"><span id="ldr">--</span></div>
    </div>
  </div>

  <div style="font-size: 0.9rem; color: #bbb; margin-bottom: 15px;">
    Status Radaru: <span id="radar" style="font-weight:bold; color:orange">-
-</span>
  </div>

```

```

<hr style="border-color: #444;">

<div class="btn-group">
  <button id="btnAuto" class="btn-auto" onclick="setMode('auto')">TRYB
AUTO</button>
  <button id="btnMan" class="btn-man" onclick="setMode('manual')">TRYB
RĘCZNY</button>
</div>

<div id="manualPanel" class="manual-controls" style="display:none;">
  <button id="mainSwitch" class="toggle-btn off"
onclick="toggleLed()">LED: WYŁĄCZONE</button>
  <label style="display:block; margin: 10px 0 5px;">Wybierz Kolor:</label>
  <input type="color" id="colorPicker" value="#ffffff"
onchange="updateColor()">
  <label style="display:block; margin: 15px 0 5px;">Jasność: <span
id="brightVal">50</span></label>
  <input type="range" min="10" max="255" value="50" id="brightSlider"
oninput="updateBrightness(this.value)"
onchange="updateBrightness(this.value)">
</div>
</div>

<script>
  let isManualOn = false;

  setInterval(getData, 1500);

  function getData() {
    fetch('/status').then(response => response.json()).then(data => {
      document.getElementById("temp").innerText = data.temp.toFixed(1);
      document.getElementById("ldr").innerText = data.ldr;
      document.getElementById("radar").innerText = data.radar ? "Wykryto Ruch"
: "Czuwanie";

      if (data.modeAuto) {
        document.getElementById("btnAuto").classList.add("active");
        document.getElementById("btnMan").classList.remove("active");
        document.getElementById("manualPanel").style.display = "none";
      } else {
        document.getElementById("btnAuto").classList.remove("active");
        document.getElementById("btnMan").classList.add("active");
        document.getElementById("manualPanel").style.display = "block";
      }
    });
  }

  function setMode(mode) {

```

```

    fetch('/set?mode=' + mode);
    setTimeout(getData, 200);
}

function toggleLed() {
    isManualOn = !isManualOn;
    let btn = document.getElementById("mainSwitch");
    if(isManualOn) {
        btn.innerText = "LED: WŁĄCZONE";
        btn.className = "toggle-btn on";
        fetch('/set?manual=1');
    } else {
        btn.innerText = "LED: WYŁĄCZONE";
        btn.className = "toggle-btn off";
        fetch('/set?manual=0');
    }
}

function updateColor() {
    var color = document.getElementById("colorPicker").value.substring(1);
    fetch('/set?color=' + color);
    if(!isManualOn) toggleLed();
}

function updateBrightness(val) {
    document.getElementById("brightVal").innerText = val;
    fetch('/set?brightness=' + val);
    if(!isManualOn) toggleLed();
}

window.onload = getData;
</script>
</body>
</html>
)rawliteral";

```

e. Opis wszystkich zmiennych.

- ssid, password – dane sieci Wi-Fi, do której łączy się ESP32.
- LED_PIN – pin sterujący taśmą WS2812B.
- RADAR_PIN – pin wejściowy podłączony do wyjścia OUT czujnika LD2410C.
- DHT_PIN – pin komunikacyjny czujnika DHT22.
- LDR_PIN – wejście analogowe do pomiaru natężenia światła.
- NUM_PIXELS – liczba diod LED w taśmie.
- CZAS_SWIECENIA – czas świecenia LED po ostatnim wykryciu ruchu.
- ostatniRuchCzas – czas ostatniego wykrycia ruchu.
- ostatniOdczytSensorow – znacznik czasu ostatniego odczytu czujników.
- trybAuto – flaga określająca tryb pracy (automatyczny/manualny).

- manualState – stan włączenia LED w trybie manualnym.
- manualColor – kolor ustawiony przez użytkownika.
- manualBrightness – jasność ustawiona przez użytkownika.
- temperatura – aktualna wartość temperatury.
- jasnocLDR – aktualny poziom oświetlenia otoczenia.
- wykrytoRuch – informacja o wykryciu obecności.

f. Opis funkcji wszystkich procedur.

- setup() – inicjalizacja pinów, czujników, taśmy LED, połączenia Wi-Fi oraz serwera WWW
- loop() – główna pętla programu, obsługa serwera, odczyt czujników oraz wybór logiki działania.
- logikaAutomatyczna() – realizuje automatyczne sterowanie LED na podstawie czujników.
- logikaManualna() – steruje LED zgodnie z ustawieniami użytkownika.
- handleControl() – obsługuje polecenia przesyłane z aplikacji webowej.
- fillStrip() – ustawia jednolity kolor na całej taśmie LED.

g. Opis interakcji oprogramowania z układem elektronicznym

Radar LD2410 (Wejście cyfrowe): mikrokontroler odczytuje stan wysoki/niski na pinie GPIO 18. Zastosowano programowy rezystor ściąający (INPUT_PULLDOWN), aby wyeliminować zakłócenia elektromagnetyczne na przewodzie sygnałowym (tzw. stany nieustalone).

Fotorezystor (Wejście analogowe): przetwornik ADC na pinie GPIO 36 zamienia napięcie (zależne od natężenia światła) na wartość cyfrową 0-4095.

DHT22 (Wejście cyfrowe): biblioteka DHT komunikuje się z czujnikiem na pinie GPIO 21, pobierając ramki danych o temperaturze.

Diody WS2812B (Wyjście cyfrowe): pin GPIO 4 generuje szybki sygnał cyfrowy, sterujący każdą diodą indywidualnie.

Komunikacja z użytkownikiem realizowana jest przez interfejs Wi-Fi i serwer WWW, który umożliwia zdalne sterowanie urządzeniem.

h. Szczegółowy opis działania ważniejszych procedur.

Kluczową procedurą jest logikaAutomatyczna(), która analizuje dane z czujników i decyduje o włączeniu lub wyłączeniu oświetlenia.

- Sprawdza stan radaru. Jeśli jest wysoki (HIGH), aktualizuje zmienną ostatniRuchCzas na bieżący czas systemowy. To mechanizm który powoduje, że światło nie zgaśnie, dopóki wykrywany jest ruch.
- Oblicza różnicę czasu (aktualnyCzas - ostatniRuchCzas). Jeśli jest mniejsza niż CZAS_SWIECENIA (10s), światło jest włączane.
- Algorytm doboru barwy: Na podstawie zmiennej temperatura wybierany jest kolor: Niebieski (<20°C), Zielony (20-26°C) lub Czerwony (>26°C).
- Algorytm doboru jasności: Funkcja map() przelicza odwrotną logikę fotorezystora (gdzie 0 to jasno, a 4095 to ciemno) na wypełnienie PWM diod (10-255). Ciemniej w pokoju = słabsze świecenie diod, aby nie oślepiać użytkownika.

5. Specyfikacja zewnętrzna:

a. Opis funkcji elementów sterujących urządzeniem.

Urządzenie nie posiada fizycznych przycisków. Sterowanie odbywa się wyłącznie poprzez Interfejs WWW dostępny w przeglądarce pod adresem IP urządzenia. Elementy interfejsu:

- Przyciski Trybu: "TRYB AUTO" i "TRYB RĘCZNY" – przełączają logikę działania.
- Przełącznik LED (w trybie ręcznym): Włącza/wyłącza oświetlenie na stałe.
- Paleta Kolorów: Pozwala wybrać dowolną barwę światła z palety RGB.
- Suwak Jasności: Płynna regulacja natężenia światła w zakresie 10-100%.

b. Opis funkcji elementów wykonawczych.

Pasek LED WS2812B: Sygnalizuje ruch, oświetla otoczenie, informuje kolorem o temperaturze otoczenia a także ma odpowiednia jasność w zależności od jasności otoczenia.

Wyświetlacz wirtualny (Panel WWW): Prezentuje dane w czasie rzeczywistym: temperaturę, poziom oświetlenia, status wykrycia ruchu oraz aktualny tryb pracy.

c. Opis reakcji oprogramowania na zdarzenia zewnętrzne.

Wykrycie ruchu powoduje włączenie oświetlenia (z opóźnieniem wynikającym jedynie z czasu reakcji pętli loop).

Brak ruchu przez określony czas powoduje wyłączenie oświetlenia.

Zmiana temperatury zmiana koloru LED.

Zmiana natężenia światła powoduje zmianę jasności LED.

Interakcja użytkownika na stronie czyli wywołanie żądania HTTP powoduje natychmiastowe przerwanie pętli oczekiwania i aktualizację parametrów (np. zmianę koloru).

d. Skrócona instrukcja obsługi urządzenia.

1. Podłącz układ do zasilacza 5 V.
2. Po włączeniu ESP32 uruchomi aplikację webową.
3. Otwórz przeglądarkę i wejdź na adres IP ESP32.
4. W panelu jest wyświetlana temperatura i jasność.
5. Wybierz tryb pracy: automatyczny lub manualny, domyślnie jest automatyczny.
6. W trybie manualnym ustaw kolor i jasność diod LED.
7. W trybie automatycznym układ pracuje samodzielnie, reagując na warunki otoczenia.

6. Opis montażu i uruchamiania.

a. Problemy które wystąpiły podczas montażu i uruchamiania i jak zostały rozwiązane.

Podczas uruchamiania prototypu napotkano następujące problemy:

Radar LD2410 wykazywał fałszywe wykrycia ruchu

Przyczyna: Przewód sygnałowy zbierał zakłócenia, dodatkowo radar był zbyt czuły.

Rozwiązanie: W kodzie programu zmieniono konfigurację pinu na INPUT_PULLDOWN (programowe ściągnięcie do masy). Dodatkowo zmniejszono czułość i czas podtrzymania radaru za pomocą dedykowanej aplikacji HLKRadarTool. Po wprowadzeniu tych zmian układ zaczął działać poprawnie.

b. Testy poprawności działania urządzenia.

Test czujnika ruchu: Wielokrotne wejście i wyjście z pola widzenia. Zmierzono czas podtrzymania (suma czasu radaru + 10s z programu) – wynik powtarzalny, ok. 15-20 sekund.

Test termostatu: Podgrzanie czujnika DHT powyżej 26°C od temperatury 18°C. Zaobserwowano zmianę koloru z niebieskiego na zielony a następnie z zielonego na czerwony. Przy ochładzaniu działanie było także poprawne.

Test optyczny: Weryfikacja działania fotorezystora polegała na symulacji skrajnych warunków oświetleniowych poprzez zasłonięcie czujnika oraz oświetlenie go silnym źródłem światła. Potwierdzono, że układ prawidłowo interpretuje zmiany natężenia światła, płynnie dostosowując jasność diod LED zgodnie z zadaniem algorytmem.

Test zdalnego sterowania: Szybkie przełączanie kolorów z poziomu telefonu. Reakcja układu była natychmiastowa.

Test stabilności: Pozostawienie układu w trybie Auto na dłuższy czas, brak samoistnych restartów czy zawieszeń.

7. Wnioski z uruchamiania i testowania.

Projekt pozwolił na stworzenie w pełni funkcjonalnego, inteligentnego układu sterowania oświetleniem. Urządzenie działa stabilnie i poprawnie reaguje na warunki otoczenia oraz polecenia użytkownika. Projekt potwierdził możliwość stworzenia kompletnego systemu IoT z wykorzystaniem ESP32, czujników oraz aplikacji webowej. Rozwiązanie może być w przyszłości rozbudowane o dodatkowe funkcje i czujniki.

Dzięki projektowi poznano praktyczne zastosowanie mikrokontrolerów rodziny ESP32 oraz różnych czujników monitorujących otoczenie.

Projekt umożliwił także zapoznanie się z programem EAGLE 9.6.2 który pozwolił na zaprojektowanie schematu układu oraz płytki PCB a także możliwość nauczenia się prawidłowego lutowania.

8. Dokumentacja końcowa linki do repozytorium GitHub oraz OneDrive:

https://polslpl-my.sharepoint.com/:u:/g/personal/ms312441_student_polsl_pl/IQBQrLTVvfDITqE24JqlsPGXAQWKgc_t2j_YSh-mz-jeS5a4?e=cmGxb3

<https://github.com/Mafiash/Projekt-SMiW-Dokumentacja-ko-cowa>